

Project Proposal

Course Title: MSDS 545 Introduction to Computational Software Engineering

Semester: Fall 2025

Students: Chadric Garrick, Megan Leak, Evann Bailey, Collin Lindsay

Title: Developing a Personal Wearable Sensor Data Analyzer Using Apple Watch Data

Introduction: Wearable devices like the Apple Watch have become increasingly important in modern health monitoring, generating rich time-series datasets such as heart rate, step count and body temperature. However, this data is not easily interpretable in its raw export format. From a software engineering perspective, transforming wearable sensor outputs into meaningful insights demonstrates critical skills in requirements gathering, system design, data pipeline implementation, testing and deployment.

Problem Statement: Apple Health data exports are provided in XML format, which is complex and not user-friendly. Individuals and researchers often lack simple tools to process and visualize this data effectively. This gap highlights the need for a robust, personal dashboard solution that applies software engineering methods to biomedical data.

Objective: The project will design and implement a web application that ingests Apple Watch data exports, cleans and processes the information, computes daily health metrics and presents the results in an interactive dashboard. This application will demonstrate full software engineering lifecycle practices while producing a useful tool for personal health analytics.

Literature Review/Background: Agile development, with its iterative cycles, is ideal for dashboard features, and the Model-View-Controller (MVC) pattern effectively separates data logic, visualization, and user interaction. While existing biomedical dashboards from apple watch are often closed systems, academic research shows that dashboards can improve personal engagement with health data. This project is valuable due to its practical application at the intersection of software engineering and biomedical informatics.

Project Description

Scope:

- **Included:** Data ingestion, cleaning, validation, metric computation (e.g., resting HR, step totals, daily averages), dashboard visualization (Streamlit), unit testing, CI/CD pipeline.
- **Excluded:** Real-time syncing from watch to app; clinical-grade diagnosis

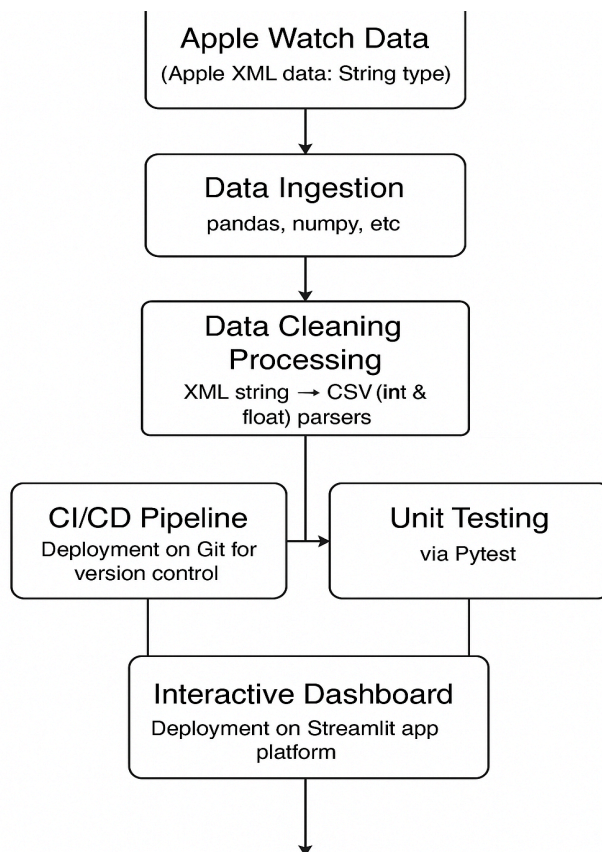
Expected Outputs:

- A working Streamlit web application.

- UML diagrams (class diagram and sequence diagram).
Documentation of methods and testing.
- GitHub repository with CI/CD enabled.

Methodology:

1. **Planning:** Define requirements from Apple Health XML format; specify key health metrics and dashboard features.
2. **Design:** Create UML diagrams for data pipeline and user interactions. Define a modular structure for the backend and the frontend.
3. **Development:**
 - Programming: Python (pandas, numpy).
 - Framework: Streamlit (dashboard).
 - Parser: XML to CSV converter for Apple Health data.
 - GitHub for version control.
4. **Testing:** Unit tests for each data processing function using pytest (Test Driven Development).
5. **Evaluation:** Assess usability (clear visuals), functionality (correct metrics), and performance (ability to process large files).



Metrics:

- Accuracy of metrics (verified against sample datasets).
- Application performance (can process large XML without failure).
- Usability feedback (ease of navigation, clarity of charts).
- Reliability (tests pass in CI/CD pipeline).

Timeline:

- **Weeks 1–2:** Research Apple Health data format, gather requirements, review related work.
- **Weeks 3–4:** Design architecture and UML diagrams.
Weeks 5–7: Development of parser, backend pipeline, and Streamlit dashboard with testing.
- **Week 8:** Final evaluation, CI/CD deployment, and presentation.

Resources Needed:

- IDE: VS Code or Google Colab
- Frameworks/Libraries: Python, pandas, numpy, Streamlit, pytest.
- Tools: GitHub
- Dataset: Apple Watch Health export (XML String Data).
- Deployment: Streamlit Community Cloud/GitHub

Expected Challenges: Large XML files can lead to performance bottlenecks, which can be mitigated through batch parsing and resampling. Timezone inconsistencies may affect daily summaries, necessitating the normalization of timestamps. Familiarity with CI/CD is required for setting up GitHub Actions, a process that can be simplified by starting with template workflows.

Conclusion: This project applies software engineering concepts to a practical biomedical data problem. By transforming Apple Watch exports into a clear, user-friendly dashboard, the work demonstrates competencies in requirements analysis, system design, development, testing and deployment. The expected outcome is a working tool that highlights both the practical utility of software engineering and its role in personal health analytics.